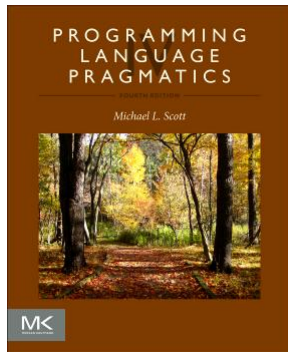


Chapter 8 :: Composite Types

Programming Language Pragmatics, Fourth Edition

Michael L. Scott



Symbol Table

- Symbol table stores information about all variables defined in the program
- The typical information kept is
 - symbol name
 - nature of the symbol (local variable, parameter, struct field etc.)
 - type of the symbol
 - scope info
 - address info (typically offset from some base address)
 - Etc.
- Various Implementations exist.
- List of Hash Tables or simply Hashing

Symbol Table Example

Index	Symbol	Type
1	void	type
2	int	type
3	getint	func: (1) $\rightarrow$ (2)
4	putint	func: (2) $\rightarrow$ (1)
5	i	(2)
6	j	(2)

```
int main() {  
    int i = getint(), j = getint();  
    while (i != j) {  
        if (i > j) i = i - j;  
        else j = j - i;  
    }  
    putint(i);  
}
```

Type Descriptor Table

- Type descriptor table stores information about all types defined in the program
- The typical information kept is
 - Name of type
 - Nature of type (struct, array, pointer etc.)
 - Name of every field and its address (in case of struct)
 - Type of object being pointed (in case of struct)
 - How many pointers are present in the type and their offsets
 - Etc.

Composite Types

- Nonscalar types are usually called composite, or constructed types.
- Created by applying a type constructor to one or more simpler types.
- Common composite types include records (structures), variant records (unions), arrays, sets, pointers, lists, and files.

Records (Structures)

- Records (structures) were introduced by Cobol, and have been supported by most languages since the 1960s.
- A record consists of collection of fields, each of which belongs to a different simpler type.

Records (Structures)

- Usually laid out contiguously
- Possible holes for alignment reasons
- Smart compilers may rearrange fields to minimize holes (C compilers promise not to)
- Implementation problems are caused by records containing dynamic arrays

Records (Structures)

C Example:

```
struct element {  
    char name[2];  
    int atomic_number;  
    double atomic_weight;  
    _Bool metallic;  
};
```

Pascal Example:

```
type two_chars = packed array  
    [1..2] of char;  
type element = packed record  
    name : two_chars;  
    atomic_number : integer;  
    atomic_weight : real;  
    metallic : Boolean  
end;
```

Records (Structures)

- Memory layout and its impact (structures)

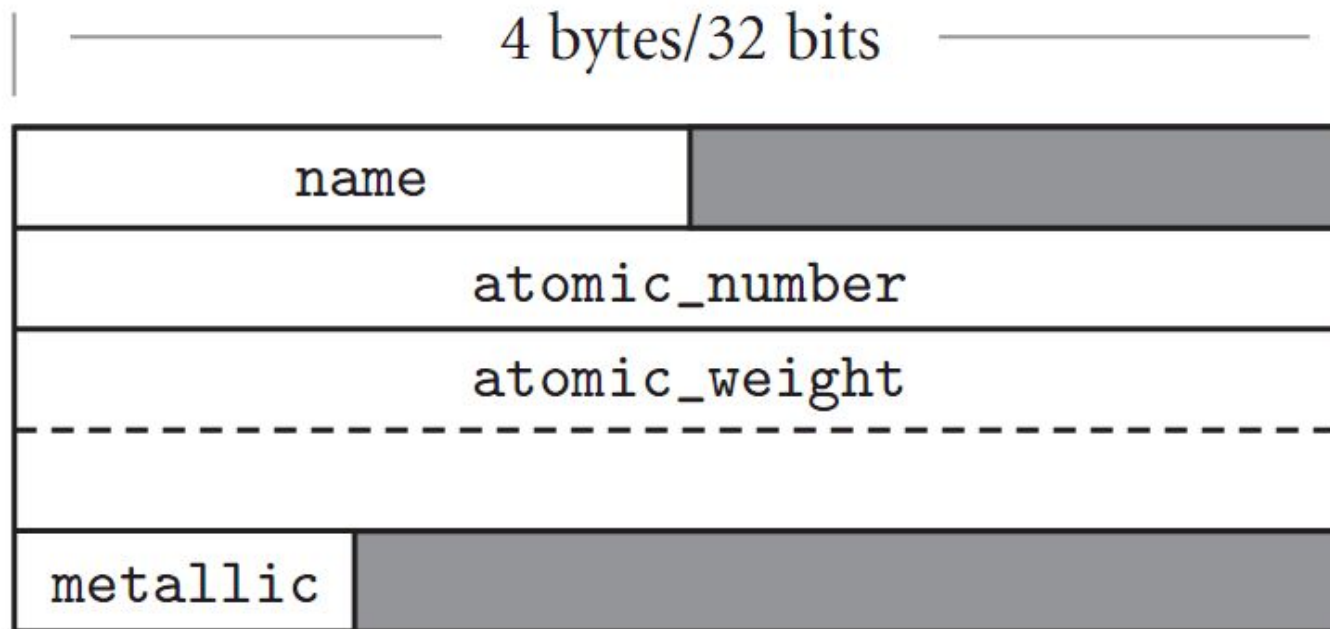


Figure 8.1 Likely layout in memory for objects of type `element` on a 32-bit machine. Alignment restrictions lead to the shaded "holes."

Records (Structures)

- Memory layout and its impact (structures)

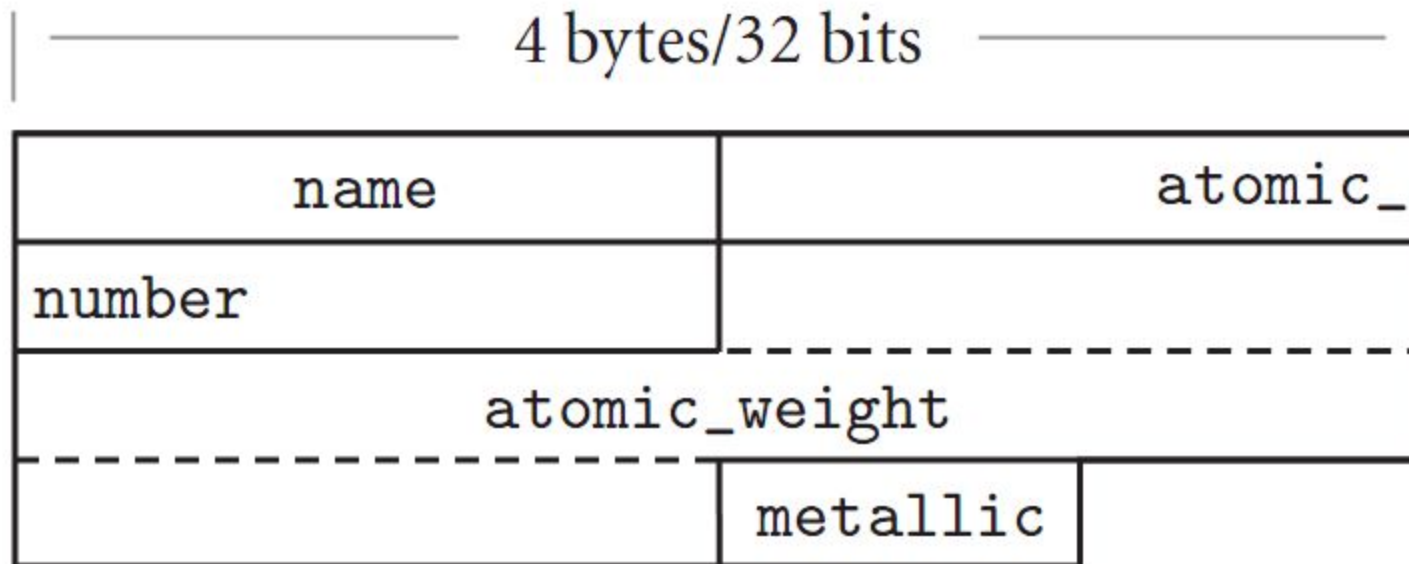


Figure 8.3 Likely memory layout for packed element records. The atomic_number and atomic_weight fields are nonaligned, and can only be read or written (on most machines) via multi-instruction sequences.

Records (Structures)

- Memory layout and its impact (structures)

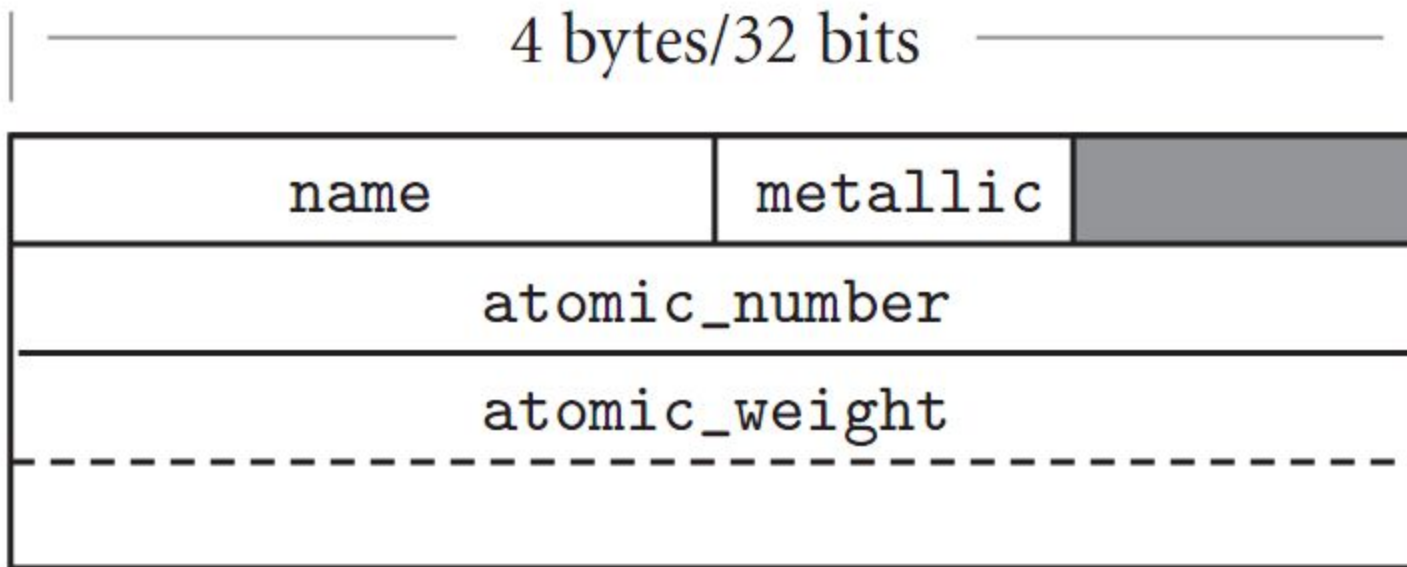


Figure 8.4 Rearranging record fields to minimize holes. By sorting fields according to the size of their alignment constraint, a compiler can minimize the space devoted to holes, while keeping the fields aligned.

Accessing a field of a record (struct)

- $x = S.f$
- Assume register $r1$ (it can be specific register like frame pointer in case a record is defined locally within the function) has starting address of struct S
- `MOV r2, offset of f in S`
 - `/* obtained from symbol table`
`Additional code is required to fetch this */`
- `MOV (x), r1(r2)`
- Once the offset has been used (considering all statements in the program), this information is not required at run-time.

Whole Record (struct) operations

- Record Copy

- Code can be generated to copy every field one at a time
(Good for small sized records)

- Comparison of two records is also done field by field

- Code can be generated to copy blocks of memory areas
from source to destination (Good for large sized
records)

- Holes (padding) bytes can have default values (zero) to enable
comparison of two records

- Compiler generates code for customized field by field
comparison for comparing two records

Unions (Variant records)

```
union { int i; float f; double d;} u;
```

```
...
```

```
u.f = 3.0;
```

```
...
```

```
printf("%f", u.f);
```

- Space allotted to the union is the space required for the maximum sized field (in this case `sizeof(double)`, say 8 bytes).
- All fields of the union share the same space.
- The programmer has to keep track of the field type that has been assigned so the same field is accessed next.

Unions (Variant records)

- Unions (variant records)
 - overlay space
 - cause problems for type checking
- Lack of tag or discriminator means we don't know what is there (some languages, like Pascal, allow a tag that is maintained by the programmer)

Variant Record in Pascal

```
type element = record
    name : two_chars;
    atomic_number : integer;
    atomic_weight : real;
    metallic : Boolean;
    case naturally_occurring : Boolean of
        true : (
            source : string_ptr;
            prevalence : real;
        );
        false : (
            lifetime : real;
        )
    end;
end;
```

Records (Structures) and Variants (Unions)

- Memory layout and its impact (unions)

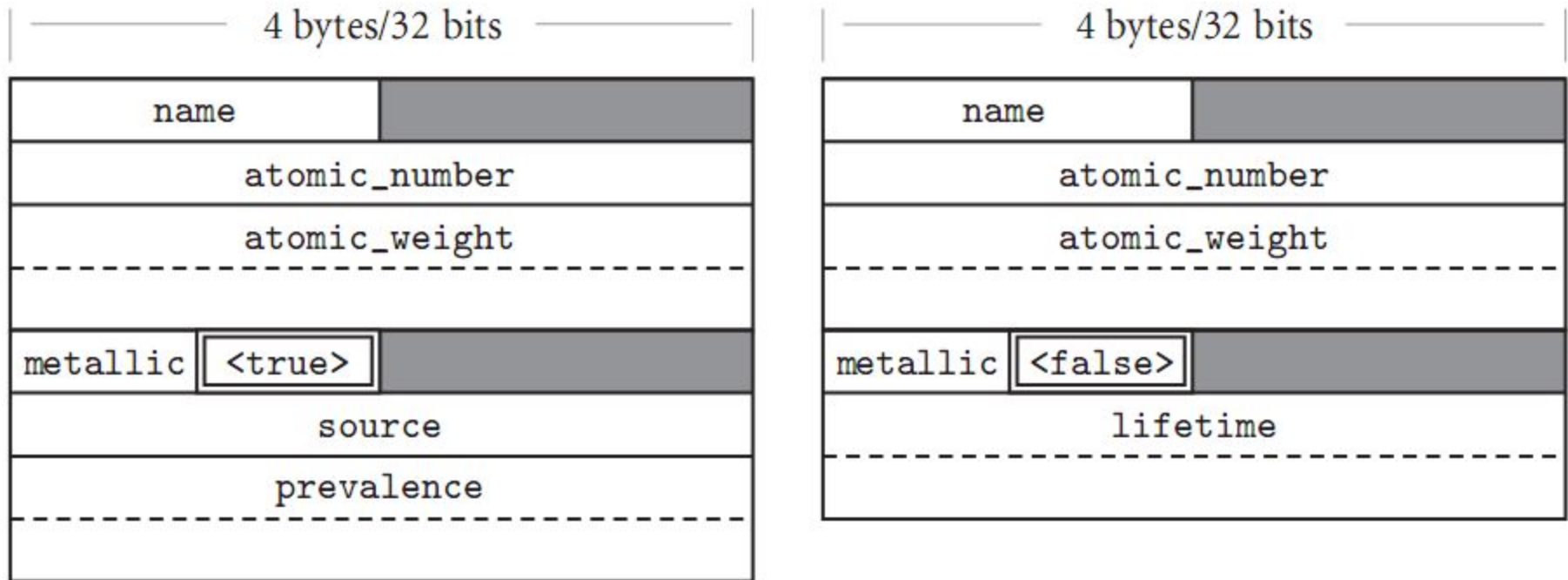


Figure 8.16 (CD) Likely memory layouts for element variants. The value of the naturally_ occurring field (shown here with a double border) is intended to indicate which of the interpretations of the remaining space is valid. Field source is assumed to point to a string that has been independently allocated.

Variants in Ada

```
type element (naturally_occurring : Boolean := true) is record
  name : string (1..2);
  atomic_number : integer;
  atomic_weight : real;
  metallic : Boolean;
  case naturally_occurring is
    when true =>
      source : string_ptr;
      prevalence : real;
    when false =>
      lifetime : real;
  end case;
end record;
```

Declaration of the discriminant of the record in its header is done, also specified a default value for it.

A declaration of a variable of type element has the option of accepting this default value:

copper : element;

or we can override it:

plutonium : element (false);

neptunium : element (naturally_occurring => false);

If the type declaration for element did not specify a default value for naturally_occurring, then all variables of type element would have to provide a value.

These rules guarantee that the tag field of a variant record is never uninitialized.

Arrays

- Arrays are the most common and important composite data types
- Unlike records, which group related fields of disparate types, arrays are usually homogeneous
- Semantically, they can be thought of as a mapping from an *index type* to a *component* or *element type*

Arrays

- Dimensions, Bounds, and Allocation
 - *global lifetime, static shape* — If the shape of an array is known at compile time, and if the array can exist throughout the execution of the program, then the compiler can allocate space for the array in static global memory
 - *local lifetime, static shape* — If the shape of the array is known at compile time, but the array should not exist throughout the execution of the program, then space can be allocated in the subroutine's stack frame at run time.
 - *local lifetime, shape bound at elaboration time*
 - *Heap??*

Arrays

```
-- Ada:  
procedure foo (size : integer) is  
  M : array (1..size, 1..size) of real;  
  ...  
begin  
  ...  
end foo;
```

```
// C99:  
void foo(int size) {  
    double M[size][size];  
    ...  
}
```

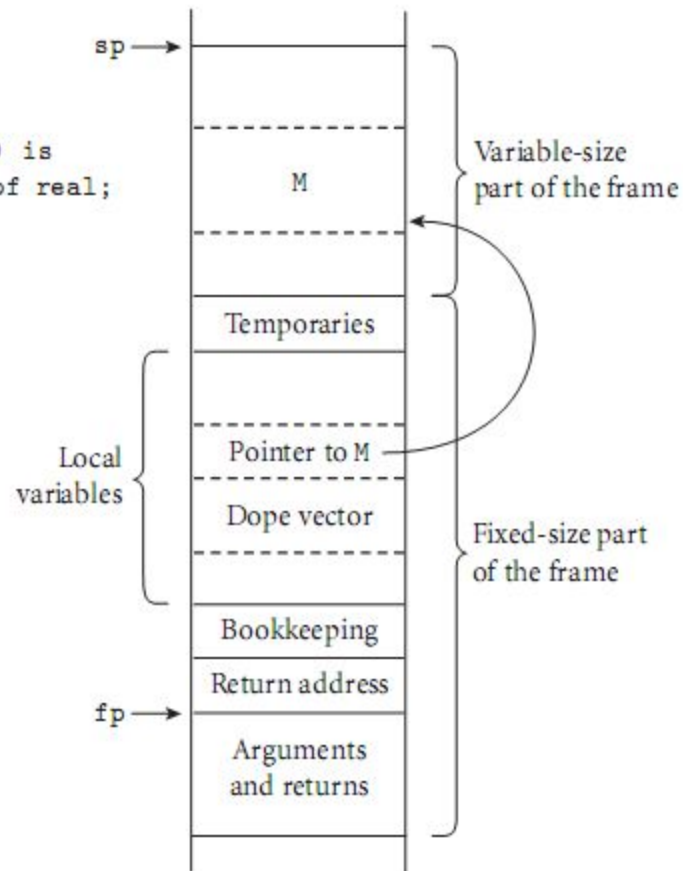


Figure 8.7 Elaboration-time allocation of arrays in Ada or C99.

Arrays

- Contiguous elements (see Figure 8.8)
 - column major - only in Fortran
 - row major
 - used by everybody else
 - makes array [a..b, c..d] the same as array [a..b] of array [c..d]

Arrays

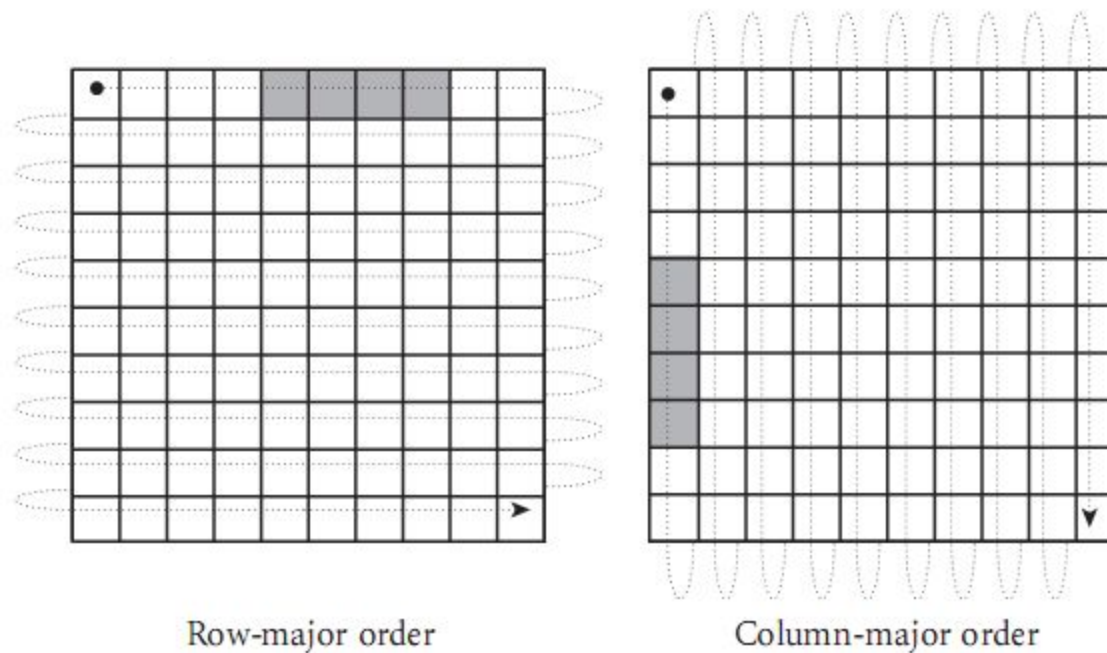


Figure 8.8 Row- and column-major memory layout for two-dimensional arrays. In row-major order, the elements of a row are contiguous in memory; in column-major order, the elements of a column are contiguous. The second cache line of each array is shaded, on the assumption that each element is an eight-byte floating-point number, that cache lines are 32 bytes long (a common size), and that the array begins at a cache line boundary. If the array is indexed from $A[0,0]$ to $A[9,9]$, then in the row-major case elements $A[0,4]$ through $A[0,7]$ share a cache line; in the column-major case elements $A[4,0]$ through $A[7,0]$ share a cache line.